

Dept. of Information Technology

Jadavpur University, Salt Lake Campus,
Block-LB/8, Sector-III, Kolkata-700106

Assignments for Object Oriented Systems Lab.

Important Instructions

1. Please sign twice (when you enter and leave the lab) in the appropriate places of the attendance sheet that will be placed on the lab instructor's table
2. All assignments given must be completed individually in the laboratory.
3. **Talking inside the lab is STRICTLY prohibited. No discussion with your classmates by any change is allowed. Get your issues (if any) resolved by the lab instructors only.**
4. Do keep your mobile phones in silent mode.
5. Use login credential (username name: user, password: user) to login to the desktop system in windows OS.
6. It is recommended to use Linux server having IP 172.16.4.112. for compilation and running your java programs. Use your login credentials to login the server. Alternatively, local computers may also be used to compile and run your java programs. (If java is not installed, install it. Password will be supplied if required)
7. It is recommended to use a simple editors like notepad, notepad++ etc. for program editing. Use terminal to compile (javac) and run (java) your programs. It is not recommended to use IDEs like Netbeans, Eclipse, IntelliJ Idea etc. If it is really required you may use VS Code.
8. To avoid getting too many diagnostic errors messages, write a few lines in your program and test. This makes debugging process easier.
9. In your solution of the assignments, write only those instructions that you understand clearly. Instructions present in your program but remains unexplained during viva voce will be considered as 'copied'.
10. Do write appropriate comments throughout your programs.
11. Save all files regularly during the lab period to avoid losing your unsaved programs/data. Create a single directory [your_roll_no] and put all of your assignments in it.
12. Use appropriate names of your program files (e.g. ass1.java, ass2.java etc.) for seamless identification of the programs corresponding to the assignments.
13. Backup your programs at appropriate places (e.g. pen drive, google drive, your linux server) as it is not guaranteed that you will get same computer every day. This will help you to quickly restore your programs.
14. Try to complete at least a few assignments from each section.

Basic syntax

1. Write a Java Program to display "Hello world".
2. Write a Java Program to print first 15 numbers in the Fibonacci series. A Fibonacci series is one whose first two numbers are 0 and 1 and any other number is obtained by adding the last two numbers. A part of the Fibonacci series is given below: 0 1 1 2 3 5 8 13 21 34 55 ...
3. Write a Java Program to store all prime numbers from 1 to 100 in an array. Now print all array elements.
4. Write a Java Program to generate first 4 rows of the Pascal triangle and store them in an array. Now print all array elements. The elements of a row of the Pascal triangle are coefficients of expansion of $(x+y)^n$. A part of Pascal triangle is:
1
1 1
1 2 1
1 3 3 1
5. Create a 3x2 array A of random integers. Create another array B such that $B=A^T$. Print both of them.
6. Create a 1-D array of 100 random integers. Now check if a given integer is in the array. (What is the complexity of your algorithm?)
7. Write a Java Program to add two 3x2 matrices (A and B) of integers and store in C. Now, print all the matrices. Initialize A as: $a[i][j] = i + j$ and B as: $b[i][j] = i*j$;

Classes & Objects

8. Create a class "Room" having two fields to store width & breadth of a room and a method to calculate the floor area of the room. Create another class "House" containing main() method which creates 2 bed rooms, 1 drawing room and 1 kitchen. Display the floor area of your house.
9. Write a class "Box" that with three fields "height", "width" and "breadth". Write suitable constructors. Also add functions like "getVolume" and "getArea" that return volume and surface area respectively. Instantiate two arbitrary boxes and then print their volume and surface area. (Can you find how many rooms of some specified size are required to store a set of given boxes?)
10. Implement a class for stack of integers. Note that the operations defined for a stack are as follows: "push", "pop", "print" etc. There should be a constructor to create a stack; with the size provided by the user. Write a "main" function to (i) create a stack to hold maximum of 10 integers; (ii) push the numbers 10, 20, 30, 15, 9 to the stack; (iii) print the stack; (iii) pop thrice and (iv) print the stack again.
11. Write a program to implement a class "student" with the following members.
 - i. Name and roll no. of the student and marks obtained in three subjects.
 - ii. Functions to compute total & averageWrite an appropriate main() method to create 4 student objects and store them in an array. Now display information of all students.
12. Write a class "Student" containing suitable member variables with suitable constructor. Write corresponding accessor/reader and modifier/writer methods. Now instantiate two-student objects and then print their details by calling proper accessor methods.
13. Write a class "BankAccount" with the following instance variables:
AccountNumber (a constant integer), balance (a floating-point number), and ownerName (a String).
Write proper constructor for this class. Also write methods balance(), add() to deposit an amount,

- subtract() to withdraw an amount and implement them. Now create another class “AccountManager” which can create, delete, debit, credit accounts. Also write a class “Bank”, add “main” function that creates an AccountManager and add 5 accounts. Now print the details of all accounts of this Bank.
14. Write a class to represent complex numbers with necessary constructors. Write member functions to add, multiply two complex numbers. There should be three constructors: (i) to initialize real and imaginary parts to 0; (ii) to initialize imaginary part to 0 but to initialize the real part to user defined value; (iii) to initialize the real part and the imaginary part to user defined values. Also, write a “main” function to (i) create two complex numbers $3+2i$ and $4-2i$; (ii) to print the sum and product of those numbers.
 15. Implement a class for “Date”. Write member functions for (i) getting the previous date, (iv) getting the next date, (iii) printing a date. There should be four constructors: (i) day, month and year are initialized to 01, 01, 1970; (ii) day is initialized to user specified value but month and year are initialized to 01, 1970; (iii) day, month are initialized to user specified value but year is initialized to 1970; (iv) day, month and year are initialized to user specified values. Also, write a “main” function to (i) create four date object using each of the constructors ; (ii) print the next and the previous date. (Should this class be final?)
 16. Implement a class for a “Book”. Book contains a title (a String), a list of authors (array of authors), number of pages (an integer), price (floating point number), publisher (a String), ISBN (a String) etc. Write suitable constructor and accessor/modifier methods. Implement a class for “Library”. A library contains a list of books. Write add (to add a book) and remove (to delete a book) methods and other relevant methods for library. Write a “main” function to create a “Library” and add a few books to the library. Print the total price of all books and no. of copies available for each book.
 17. Write a Java class “Employee” containing name, id, address, salary etc. Write necessary constructor and reader/writer methods. Create a class “Dept” that has a name, location, year of establishment etc. The “Dept” contains a number of “Employee”. Write methods “add” and “remove” to add and remove an employee to/from this department. Write a “main” function and create “Information Technology” department. Add five employees and print annual salary to be paid to all employees of this department.
 18. Implement a class for a “Student”. Information about a student includes name, roll no and an array of five subject names. The class should have suitable constructor and get/set methods. Implement a class “TabulationSheet”. A tabulation sheet contains roll numbers and marks of each student for a particular subject. This class should have a method for adding the marks and roll no of a student. Implement a class “MarkSheet”. A mark sheet contains marks of all subjects for a particular student. This class should have a method to add name of a student and marks in each subject. Write a “main” function to create three “Student” objects, Five “Tabulationsheet” objects for Five subjects and three “Marksheet” object for three students. Print the mark sheets.

Inheritance

19. Create an abstract class “Publication” with data members ‘noOfPages’, ‘price’, ‘publisherName’ etc. with their accessor/modifier functions. Now create two sub classes “Book” and “Journal”. Create a class Library that contains a list of Publications. Write a main function and create three Books and two Journals and add them to Library and print the details of all publications.
20. Write a class for “Account” containing data members ‘accountNumber’, ‘holderName’, ‘balance’ and add constructors and necessary accessor/modifier functions for these data members. Now create two class “SavingsAccount” and “CurrentAccount” extending from this class. SavingsAccount will have a member variable ‘interestRate’ and member function ‘calculateYearlyInterest’. Write another class “Manager” that contains a list Account. Also write a main function to create an instance of Manager class. Add two SavingsAccount and three CurrentAccount to Manager. Calculate interest of each SavingsAccount. Print the details of all accounts.

21. Implement a class for a "Person". Person has data members 'age', 'weight', 'height', 'dateOfBirth', 'address' with proper reader/write methods etc. Now create two subclasses "Employee" and "Student". Employee will have additional data member 'salary', 'dateOfJoining', 'experience' etc. Student has data members 'roll', 'listOfSubjects', their marks, and methods 'calculateGrade'. Again create two sub-classes "Technician" and "Professor" from Employee. Professor has data members 'courses', 'listOfAdvisee' and their add/remove methods. Write a main function to demonstrate the creation of objects of different classes and their method calls.
22. Create a base class "Automobile". An Automobile contains data members 'make', 'type', 'maxSpeed', 'price', 'mileage', 'registrationNumber' etc. with their reader/writer methods. Now create two sub classes "Track" and "Car". Track has data members 'capacity', 'hoodType', 'noOfWheels' etc. Car has data member 'noOfDoors', 'seatingCapacity' and their reader/write methods. Create a main function to demonstrate this.

Interface

23. Create an interface "Shape" that contains methods 'area', 'draw', 'rotate', 'move' etc. Now create two classes "Circle" and "Rectangle" that implement this 'Shape' interface and implement the methods 'area', 'move', etc. Write a main function to create two "Circle" and three "Rectangle" the move them. Print the details of circles and rectangles before after moving them. (What is the purpose of Shape interface? What is the difference between an abstract class and an interface?)
24. Write two interfaces "Fruit" and "Vegetable" containing methods 'hasAPeel' and 'hasARoot'. Now write a class "Tomato" implementing Fruit and Vegetable. Instantiate a Tomato object. Print the details of this object.

Exception Handling

25. Write a code segment in a try block where divide by zero occurs such as 'int x=0, y=3/x'; Also write corresponding 'catch' block to catch the exception that occurs in the try block. Print the origin of the exception caught.
26. Create an array of 10 integers and try to assign an integer in location 15 of the array. Put it in a try-catch block. Print the appropriate message in the catch block.
27. Consider the previous assignment. Write two catch blocks, one for catching "Exception" another for catching "ArrayIndexOutOfBoundsException". In the first catch block re-throw the exception caught. In the second catch block print the origin of the exception recursively.
28. Create a superclass "MathException" and two subclasses "OverflowException" and "UnderFlowException" from it. Write code segment that throws an OverflowException. Write three catch blocks, one for MathException, another for OverflowException and another for UnderFlowException. In the first catch block re-throw the exception caught. In the other two catch blocks write appropriate statements to handle it. Observe the result.
29. Consider the previous assignment. Instantiate an integer variable and initialize it with some value. If the value is greater than 100, an OverflowException is thrown, otherwise an UnderFlowException is called. Write appropriate message in the catch blocks and observe the result.
30. Write a code segment that results a "NullPointerException". Write necessary catch block to catch it. Also write a 'finally block'. Write appropriate statements in it.

Package and Nested Classes:

31. Place your classes under appropriate packages. For example, inheritance and exception related classes may be placed in java.[your_roll_no].inheritance & java.[your-roll_no].exception packages respectively. Now, compile them and use them from a different package. Observe the result.

I/O

32. Write a program to read elements of an nxm matrix through keyboard and print it.
33. Read a file and count the number of words in the file. A word is delimited by either a space or a tab or a new line or end-of-file or a punctuation symbol (“,”, “!”, “.”, “?”, “;”).
34. Read a file and prepare a table that contains information about how many times a word occurs in the file. Each row contains a word that occurs in the file and a number that denotes the number of times this word occurs in the file. Your program has to print this table.

Thread

35. Write a program to create two threads. Print “In main thread” in main thread and “In child thread” in child thread.
36. Consider the series $1+2+3+\dots+100$. This can be considered as $(1+3+5+\dots+99)+(2+4+6+\dots+100)$. Create two threads to compute two series in parallel (do not use simplified equation). Lastly print the final sum.
37. Consider the following series $x = 1 + 1/1! + 1/2! + 1/3! + \dots + 1/10!$
Create two threads t1 & t2. t1 will generate the denominators and t2 will form the term and add them up. Finally print the result.
38. Create two threads and call them EvenThread and OddThread. EvenThread will print number as 2 4 6 8 10... and OddThread will print number as 1 3 5.... Now synchronize these two threads to get the output as 1 2 3 4 5 6 7 8.
39. Consider a file that contains a number of integers. Create two threads ‘producer’ and ‘consumer’. Producer thread reads the integers from the file continuously while consumer thread adds them up. Print the final value. Use proper synchronization mechanism if needed.
40. Consider the following parallel binary search algorithm for series $a_0, a_1 \dots a_{n-1}$ sorted in increasing order such that $n \bmod 10 = 0$. Element to be searched is e.
 - a) Create $n/10$ threads $t_0, t_1, \dots, t_{n/10-1}$.
 - b) Distribute the numbers among threads such that t_i will have numbers $a_{10i}, a_{10i+1}, \dots, a_{10(i+1)-1}$.
 - c) Distribute the element e to all threads.
 - d) Each thread searches the element e in its sub-array using binary search algorithm.Write a Java program using threading technology and print the thread index and location where the element has been found.

Generic Programming:

41. Write a generic method to exchange the positions of two different elements in an array. Test your method with two different arrays.
42. Write a generic stack class. Now create two stacks of 5 integers and strings respectively. Push 5 elements and pop twice. Print stack elements after pushing and popping.

Reflection

43. Write a program to display information of String class such as package it contained it, synthetic or not, modifiers, interface implemented, superclass name, declared fields and their modifiers, methods and their modifiers etc. using Java reflection API.
44. Write a program to invoke “length” on a String object using conventional way and reflection. Compare the output. (What is the advantage of reflection? As their any drawback of reflection?)

RMI

45. Implement a class “Calculator” having methods add(), subtract() etc. Write one program that create an instance of Calculator object and another program (may run in different computer) that invokes add() method on this object using RMI technology. (What is was the purpose of interface here?)
46. Write a java program to demonstrate object migration from one computer to another using RMI and Object stream.
47. Write a program to implement parallel binary search using RMI & thread technology. In parallel binary search, segments of a sorted array and element to be searched are distributed to different computers. These computers run the search algorithm on given data using local processor(s) and return the result.

UML

48. Draw UML class and sequence diagrams for a few programs that you have written earlier.
[You may use online tools: diagrams.net, lucid chart or offline tools: VisualParadigm, UML Lab, StarUML etc.]

Design Patterns

49. Write a java program to demonstrate Singleton design pattern. (Creational)(How efficient is your design?)
50. Write a java program to demonstrate Adapter design pattern. (Structural)
51. Write a java program to demonstrate Iterator design pattern. (Behavioral)